



16

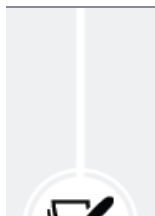
Foundation Models in Computer Vision

In the previous chapter, we learned about how we can build novel applications using NLP and CV techniques. However, this requires a significant amount of training, either from scratch or by fine-tuning a pre-trained model. When leveraging a pre-trained model, the model has generally been trained on a large corpus of data – for example, a dataset like ImageNet, which contains ~21 million images. However, on the internet, we have access to hundreds of millions of images and the alt text corresponding to those images. What if we pre-train models on internet-scale data and use those models for different applications involving object detection, segmentation, and text-to-image generation out of the box without any fine-tuning? This forms the bedrock of foundation models.

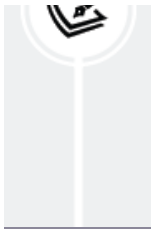
In this chapter, we will learn about:

- Leveraging image and text embeddings to identify the most relevant image for a given text and vice versa
- Leveraging image and text encodings to perform zero-shot image object detection and segmentation
- Building a diffusion model from scratch, using which we'll generate images both conditionally (with text prompting) and unconditionally
- Prompt engineering to generate better images

More specifically, we will learn about **Contrastive Language-Image Pre-training (CLIP)**, which can identify images relevant to a given text and vice versa, combining an image encoder and prompt (text) encoder to identify regions/segments within an image. We'll learn how to leverage CNN-based architectures with the **Segment Anything Model (SAM)** to perform zero-shot segmentation ~50X faster than transformer-based zero-shot segmentation. We will also learn about the fundamentals of **Stable Diffusion** models and the XL variant.



All code snippets within this chapter are available in the `Chapter16` folder of the GitHub repository at <https://bit.ly/mcvp-2e>.



As the field evolves, we will periodically add valuable supplements to the GitHub repository. Do check the `supplementary_sections` folder within each chapter's directory for new and useful content.

Introducing CLIP

Imagine a scenario where you have access to an image dataset. You are not given the labels corresponding to each image in the dataset. However, you have the information in the form of an exhaustive list of all the unique labels present in the image dataset. How would you assign the probable label for a given image?

CLIP comes to the rescue in such a scenario. CLIP provides an embedding corresponding to each image and label (text associated with the image – typically, the class of image or the caption of the image). This way, we can associate an image with the most probable label – where the similarity of image embeddings and the text embeddings of all the unique labels are calculated and the label with the highest similarity to the image embeddings is the most probable label.

In the next section, let us get an understanding of CLIP and build a CLIP model from scratch before we use a pre-trained one.

How CLIP works

To understand how CLIP works, let us de-construct the acronym **CLIP** (**Contrastive Language-Image Pre-training**):

- Pre-training: The model is trained on a huge corpus of data.
- Language-Image Pre-training: The model is trained on a huge corpus of data that contains both images and the text corresponding to them – for example, the alt-text corresponding to an image.
- Contrastive Language-Image Pre-training: The language-image pre-training is done in such a way that the image and text embeddings of similar images will be similar while the image embedding of one object and the text embedding corresponding to another object is as dissimilar as possible.

Let us understand this with the following diagram:

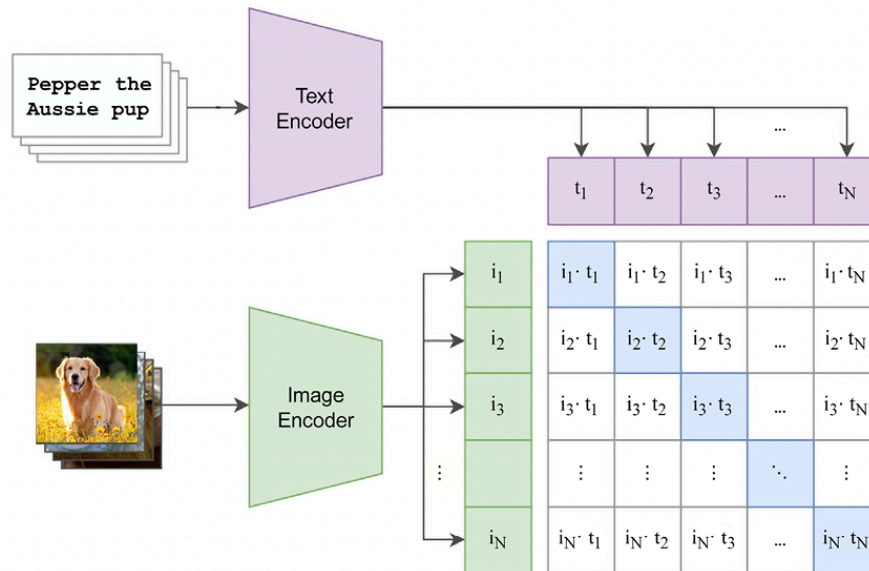


Figure 16.1: Overview of CLIP

In the preceding diagram, we have a set of images and texts. Further, we have the embeddings ($i_1, i_2, \dots, i_N$) that are obtained once these images are passed through the image encoder and a set of corresponding text embeddings ($t_1, t_2, \dots, t_N$) that are obtained once the texts are passed through the text encoder.

We now need to train the model in such a way that i_N has a high similarity with t_N and has very low similarity with text embeddings corresponding to other images.

In the next section, let us go ahead and train the CLIP model from scratch.

Building a CLIP model from scratch

To build a CLIP model from scratch, we will leverage the FLICKR-8K dataset as it contains both the images and the associated captions:



The following code is available in the `CLIP_from_scratch.ipynb` file in the `Chapter16` folder in the GitHub repo at <https://bit.ly/mcvcv-2e>. Do be sure to run the code from the notebook and refer to the explanations provided here to understand the different steps. You will need a Kaggle account to download the data.

1. Install the required packages and import the required modules:

```
%%capture
import os
if not os.path.exists("MCVP2e-CLIP"):
    !git clone https://github.com/sizhky/MCVP2e-CLIP.git
    %pip install -r MCVP2e-CLIP/requirements.txt
%cd MCVP2e-CLIP
```

2. Import the required packages:

```
import itertools
from torch_snippets import *
from clip.core import download_flickr8k_from_kaggle
from clip.config import ClipConfig
from clip.dataset import CLIPDataset
from clip.models import CLIP
```

3. Provide your Kaggle credentials:

```
%%writefile kaggle.json
{"username": "XXXX", "key": "XXXX"}
```

4. Download the Flickr dataset. Before running the next line of code, ensure you agree to Kaggle's terms on the webpage at <https://www.kaggle.com/datasets/adityajn105/flickr8k>, otherwise, the download won't happen.



```
kaggle_json_path = P("kaggle.json")
data_download_path = P("/content/flickr-8k-kaggle/")
download_flickr8k_from_kaggle(kaggle_json_path, data_download_path)
df = pd.read_csv(data_download_path / "captions.txt")
df["id"] = [id_ for id_ in range(len(df) // 5) for _ in range(5)]
df.to_csv(data_download_path / "captions.csv")
```

5. Set up your training configuration:

```
config = ClipConfig()
config.image_path = data_download_path / "Images"
config.captions_csv_path = data_download_path / "captions.csv"
# Make any other experiment changes, if you want to, below
config.debug = False # Switch to True, in case you want to reduce the dataset siz
```

```
config.epochs = 1
config.save_eval_and_logging_steps = 50
```

You are advised to go through the full `ClipConfig` class in the notebook on GitHub and understand the individual parameters – the learning rates of the image encoder and text encoder, epochs, the backend model name, the image and text embeddings, the maximum sequence length of text, and the dimensions of the projection layer (as the embedding dimensions of images and text are different). For now, we are simply pointing to the right dataset paths and setting the number of epochs.

6. Create the training and validation datasets.

The key components of the dataset are given below (the code is imported from the `dataset.py` script).

```
class CLIPDataset(Dataset):
    def __init__(self, df, config, mode):
        """
        image_filenames and captions must have the same length; so, if there are
        multiple captions for each image, the image_filenames must have repetitive
        file names
        """
        self.config = config
        self.tokenizer = DistilBertTokenizer.from_pretrained(
            config.distilbert_text_tokenizer
        )
        self.image_filenames = df.image.tolist()
        self.captions = df.caption.tolist()
        with notify_waiting(f"Creating encoded captions for {mode} dataset..."):
            self.encoded_captions = self.tokenizer(
                self.captions,
                padding=True,
                truncation=True,
                max_length=config.max_length,
            )
        self.transforms = get_transforms(config)
    def __getitem__(self, idx):
        item = {
            key: torch.tensor(values[idx])
            for key, values in self.encoded_captions.items()
        }
        image = \
read(f"{self.config.image_path}/{self.image_filenames[idx]}", 1)
        image = self.transforms(image=image)
        item["image"] = torch.tensor(image).permute(2, 0, 1).float()
```

```

        item["caption"] = self.captions[idx]
    return item
def __len__(self):
    return len(self.captions)

```

All text is tokenized using the DistilBert tokenizer. All the images are transformed to a fixed size and are normalized using the `get_transforms` method. We've omitted the explanation of the other methods used in the `Dataset` class for brevity. But be sure to inspect the class and understand the different methods defined in the `dataset.py` script.

Use the following snippet to create the training and validation datasets:

```
trn_ds, val_ds = CLIPDataset.train_test_split(config)
```

7. Load the model:

```
model = CLIP(config).to(config.device)
```

8. Let's look at the key components of the model:

1. The `ImageEncoder` class: Using the `ImageEncoder` class, we fetch embeddings of the image, which are obtained by passing the image through a resnet-50 pre-trained model:

```

class ImageEncoder(nn.Module):
    """
    Encode images to a fixed size vector
    """
    def __init__(
        self, model_name=CFG.model_name,
            pretrained=CFG.pretrained,
            trainable=CFG.trainable):
        super().__init__()
        self.model = timm.create_model(
            model_name, pretrained, num_classes=0,
                                   global_pool="avg")
        for p in self.model.parameters():
            p.requires_grad = trainable
    def forward(self, x):
        return self.model(x)

```

2. `TextEncoder` converts the tokens into an embedding by passing the text (tokens) through a BERT model.

```

class TextEncoder(nn.Module):
    def __init__(self, model_name=CFG.text_encoder_model,
                  pretrained=CFG.pretrained, trainable=CFG.trainable):
        super().__init__()
        if pretrained:
            self.model = DistilBertModel.from_pretrained(model_name)
        else:
            self.model = DistilBertModel(config=DistilBertConfig())

        for p in self.model.parameters():
            p.requires_grad = trainable
        # we are using CLS token hidden representation as
        # the sentence's embedding
        self.target_token_idx = 0
    def forward(self, input_ids, attention_mask):
        output = self.model(input_ids=input_ids,
                             attention_mask=attention_mask)
        last_hidden_state = output.last_hidden_state
        return last_hidden_state[:, self.target_token_idx, :]

```

3. **ProjectionHead**: This is needed as the text encoder output is 768-dimensional while the image encoder is 2048-dimensional. We need to bring them to the same dimensionality to perform a comparison.

```

class ProjectionHead(nn.Module):
    def __init__(
        self,
        embedding_dim,
        projection_dim=CFG.projection_dim,
        dropout=CFG.dropout
    ):
        super().__init__()
        self.projection = nn.Linear(embedding_dim, projection_dim)
        self.gelu = nn.GELU()
        self.fc = nn.Linear(projection_dim, projection_dim)
        self.dropout = nn.Dropout(dropout)
        self.layer_norm = nn.LayerNorm(projection_dim)

    def forward(self, x):
        projected = self.projection(x)
        x = self.gelu(projected)
        x = self.fc(x)
        x = self.dropout(x)
        x = x + projected
        x = self.layer_norm(x)
        return x

```

9. Build the `CLIPModel`. To do this, we obtain `image_embedding` and `text_embedding` of the same dimensionality. Next, we calculate logits to understand the similarity between text and image embeddings. Then, we calculate the similarity between the embedding of a given image with the embeddings of the remaining images (similarly for text). Finally, we calculate the overall loss with the intuition that similar images will have similar embeddings and dissimilar images will have embeddings that are far away:

```
class CLIPModel(nn.Module):
    def __init__(
        self,
        temperature=CFG.temperature,
        image_embedding=CFG.image_embedding,
        text_embedding=CFG.text_embedding,
    ):
        super().__init__()
        self.image_encoder = ImageEncoder()
        self.text_encoder = TextEncoder()
        self.image_projection = \
            ProjectionHead(embedding_dim=image_embedding)
        self.text_projection = \
            ProjectionHead(embedding_dim=text_embedding)
        self.temperature = temperature
    def forward(self, batch):
        # Getting Image and Text Features
        image_features = self.image_encoder(batch["image"])
        text_features = self.text_encoder( \
            input_ids=batch["input_ids"],
            attention_mask=batch["attention_mask"])
        # Getting Image and Text Embeddings (with same dimension)
        image_embeddings= self.image_projection(image_features)
        text_embeddings = self.text_projection(text_features)
        # Calculating the Loss
        logits = (text_embeddings @ image_embeddings.T)/self.temperature
        images_similarity = image_embeddings @ image_embeddings.T
        texts_similarity = text_embeddings @ text_embeddings.T
        targets = F.softmax( (images_similarity + texts_similarity) / 2 * \
            self.temperature, dim=-1)
        texts_loss = cross_entropy(logits, targets, reduction='none')
        images_loss = cross_entropy(logits.T, targets.T, reduction='none')
        loss = (images_loss + texts_loss) / 2.0 # shape: (batch_size)
        return {"loss": loss.mean()}
```

The loss function used in the preceding code is as follows:


```
def cross_entropy(preds, targets, reduction='none'):
    log_softmax = nn.LogSoftmax(dim=-1)
    loss = (-targets * log_softmax(preds)).sum(1)
    if reduction == "none":
        return loss
    elif reduction == "mean":
        return loss.mean()
```

We return the output as a dictionary with the `loss` key to keep it compatible with Hugging Face.

10. Load the optimizer and define the learning rate for the image encoder and text encoder.

```
params = [
    {"params": model.image_encoder.parameters(), "lr":
                                     config.image_encoder_lr},
    {"params": model.text_encoder.parameters(), "lr":
                                     config.text_encoder_lr},
    {
        "params": itertools.chain(
            model.image_projection.parameters(),
            model.text_projection.parameters()
        ),
        "lr": config.head_lr,
        "weight_decay": config.weight_decay,
    },
]
optimizer = torch.optim.AdamW(params, weight_decay=0.0)
lr_scheduler=torch.optim.lr_scheduler.ReduceLROnPlateau( \
                                     optimizer, mode="min",
                                     patience=config.patience,
                                     factor=config.factor)
```

11. Train with the Hugging Face API:

```
from transformers import Trainer, TrainingArguments
# Define TrainingArguments
training_args = TrainingArguments(
    output_dir="./results", # Output directory where checkpoints and logs will be
    num_train_epochs=config.epochs, # Total number of training epochs.
    per_device_train_batch_size=config.batch_size, # Batch size per GPU.
    per_device_eval_batch_size=config.batch_size, # Batch size for evaluation
    evaluation_strategy="steps", # Evaluation strategy (steps, epoch).
    logging_strategy="steps", # Logging strategy (steps, epoch).
    save_strategy="steps", # Save strategy (steps, epoch).
```

```

save_total_limit=2, # Limit the total amount of checkpoints.
learning_rate=5e-5, # Learning rate.
logging_steps=config.save_eval_and_logging_steps,
save_steps=config.save_eval_and_logging_steps, # Save checkpoints every N #step
eval_steps=config.save_eval_and_logging_steps, # Evaluate every N steps.
logging_dir="./logs", # Directory for storing logs.
metric_for_best_model="loss",
label_names=["image", "input_ids"],
)
# Create Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=trn_ds,
    eval_dataset=val_ds,
    optimizers=(optimizer, lr_scheduler),
)
# Train the model
trainer.train()

```

Note

Similar to the detectron trainer, mmaction2 runner, we have a Hugging Face trainer that is a wrapper around `opt.zero_grad()`, `loss.backward()`, etc., steps to keep the training code concise for the end user. Studying the actual trainer class in depth is left as an exercise for you.

12. Once the training is done, we can fetch embeddings corresponding to all the images:

```

def get_image_embeddings(valid_df, model_path):
    tokenizer = DistilBertTokenizer.from_pretrained(CFG.text_tokenizer)
    valid_loader = build_loaders(valid_df, tokenizer, mode="valid")

    model = CLIPModel().to(CFG.device)
    model.load_state_dict(torch.load(model_path,
                                     map_location=CFG.device))
    model.eval()

    valid_image_embeddings = []
    with torch.no_grad():
        for batch in tqdm(valid_loader):
            image_features = \
                model.image_encoder(batch["image"].to(CFG.device))
            image_embeddings = \

```

```

        model.image_projection(image_features)
        valid_image_embeddings.append(image_embeddings)
    return model, torch.cat(valid_image_embeddings)
_, valid_df = make_train_valid_dfs()
model, image_embeddings = get_image_embeddings(valid_df, "best.pt")

```

13. Find matches to a given query (user input text):

```

def find_matches(model, image_embeddings, query, image_filenames, n=9):
    tokenizer = DistilBertTokenizer.from_pretrained(CFG.text_tokenizer)
    encoded_query = tokenizer([query])
    batch = {
        key: torch.tensor(values).to(CFG.device)
        for key, values in encoded_query.items()
    }
    with torch.no_grad():
        text_features = model.text_encoder(
            input_ids=batch["input_ids"],
            attention_mask=batch["attention_mask"]
        )
        text_embeddings = model.text_projection(text_features)

    image_embeddings_n = F.normalize(image_embeddings, p=2, dim=-1)
    text_embeddings_n = F.normalize(text_embeddings, p=2, dim=-1)
    dot_similarity = text_embeddings_n @ image_embeddings_n.T

    values, indices = torch.topk(dot_similarity.squeeze(0), n * 5)
    matches = [image_filenames[idx] for idx in indices[:5]]
    _, axes = plt.subplots(3, 3, figsize=(10, 10))
    for match, ax in zip(matches, axes.flatten()):
        image = cv2.imread(f"{CFG.image_path}/{match}")
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        ax.imshow(image)
        ax.axis("off")
    plt.show()
find_matches(model,
            image_embeddings,
            query="dogs on the grass",
            image_filenames=valid_df['image'].values,
            n=9)

```

The preceding code results in an output, as follows:



Figure 16.2: Images matching the query “dogs on the grass”

With this, we are now able to provide a matching image for a given query.

Now that we understand how CLIP is trained from scratch, we will learn about leveraging the OpenAI API to get embeddings corresponding to images and texts.

Leveraging OpenAI CLIP

One of the problems with training CLIP from scratch is that it would take considerable resources – both compute as well as the data required to train the model. **OpenAI CLIP** is already trained on a dataset of 400 million image text pairs. It is queried using the following steps:



The following code is available in the `OpenAI_CLIP.ipynb` file in the `Chapter16` folder in the GitHub repo at <https://bit.ly/mcyp-2e>. Do be sure to run the code from the notebook and refer to the following explanations to understand the different steps.

1. Install the required packages:

```
!pip install ftfy regex tqdm
```

2. Clone the GitHub repository:

```
!git clone https://github.com/openai/CLIP.git
%cd CLIP
```

3. Load the pre-trained CLIP model:

```
import torch
import clip
from PIL import Image
device = "cuda" if torch.cuda.is_available() else "cpu"
model, preprocess = clip.load("ViT-B/32", device=device)
```

4. Provide an image and text and pre-process them:

```
image = preprocess(Image.open("CLIP.png")).unsqueeze(0).to(device)
text=clip.tokenize(["a diagram", "a dog", "a cat"]).to(device)
```

In the preceding code, we provide an image of a diagram and the labels that possibly correspond to the image.

The image we provided is:

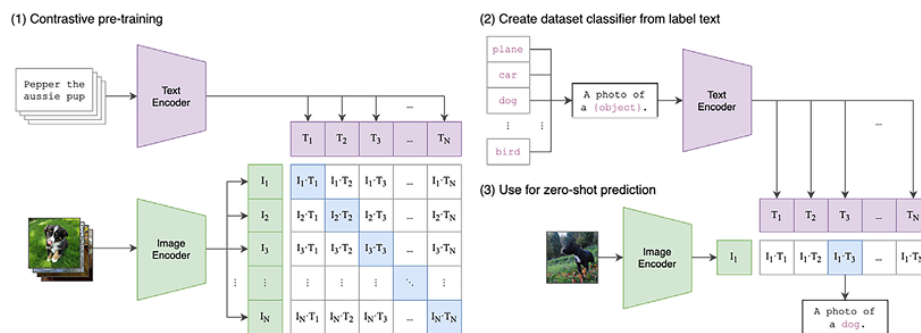


Figure 16.3: Input image

5. Pass the image and the text embeddings through the feed-forward network:

```
with torch.no_grad():
    image_features = model.encode_image(image)
```

```
text_features = model.encode_text(text)

logits_per_image, logits_per_text = model(image, text)
probs = logits_per_image.softmax(dim=-1).cpu().numpy()
print("Label probs:", probs)
# prints: [[0.9927937  0.00421068  0.00299572]]
```

With the above, we can get the corresponding label, given an image and the set of possible labels.

Now that we understand how to get a label in a zero-shot setting with the constraint that the set of possible labels is provided, in the next section, we will learn about segmenting an image and fetching the relevant content (only the mask corresponding to the label) by specifying the label.

Introducing SAM

Imagine a scenario where you are given an image and are asked to predict the mask corresponding to a given text (let's say a dog in an image where there are multiple objects, like a dog, cat, person, and so on). How would you go about solving this problem?

In a traditional setting, this is an object detection problem where we need data to perform fine-tuning on a given dataset or leverage a pre-trained model. We are unable to leverage CLIP as it classifies the overall picture and not individual objects within it.

Further, in this scenario, we want to do all of this without even training a model. Here is where **Segment Anything Model (SAM)** - <https://arxiv.org/pdf/2304.02643> from Meta helps in solving the problem.

How SAM works

SAM is trained on a corpus of 1 billion masks generated from 11 million images. These 1 billion images (SAM 1B dataset) are from the data engine that Meta developed in the following stages:

1. Assisted manual – SAM assists annotators in annotating masks
2. Semi-automatic – SAM generates masks for a subset of objects present in an image, while annotators mask the remaining objects present in the image
3. Fully automatic – Humans prompt SAM with a grid of points and SAM automatically generates masks corresponding to the points

Using these three steps, one can generate a greater number of masks per image (as SAM prompts for small objects that humans might miss).

The preceding steps result in a considerably high number of masks per image, as follows:

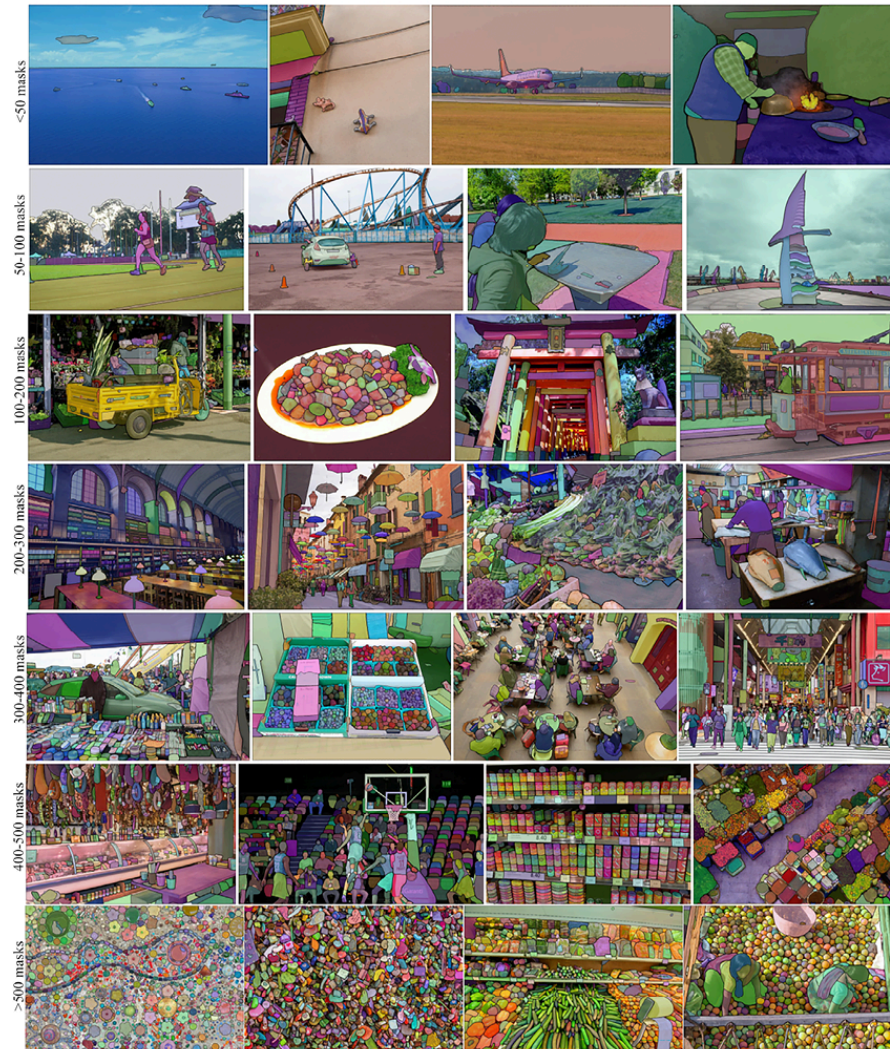


Figure 16.4: Masks generated using SAM

The architecture of SAM is as follows:

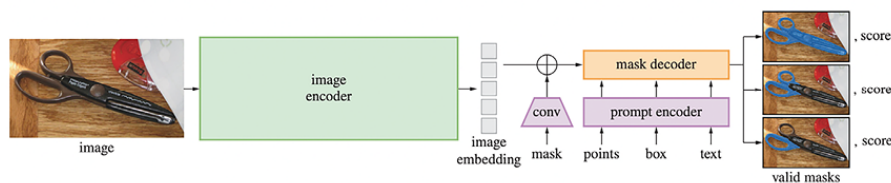


Figure 16.5: SAM architecture

In the preceding image, the following steps are being performed:

1. An image is passed through an image encoder (which is a pre-trained vision transformer) to get the corresponding image embeddings.
2. Once we have the image embeddings, we can query for specific masks in the overall image by providing the dense mask (mask in the above figure) or sparse masks (points, boxes, and text).
3. The prompt encoder is responsible for taking in points, boxes, and masks, and returning dense and sparse embeddings. Point and box embeddings are constructed in a manner similar to how we calculated embeddings in LayoutLM. Let's look at the differences between different types of masks:
 1. Dense masks are useful in scenarios where we want to segment the hair of a person or leaves in a tree, where we provide the rough contour of the mask that we want to extract, and SAM does the job of extracting the mask.
 2. Sparse masks are useful when we want to specify the text/bounding box/point corresponding to the image.
 3. If we don't provide any mask, text, or point input, the model will auto-generate 1,024 points uniformly across the input image for the point encoder.

At a high level, the `forward` method for the prompt encoder looks like so:

```
def forward(
    self,
    points: Optional[Tuple[torch.Tensor, torch.Tensor]],
    boxes: Optional[torch.Tensor],
    masks: Optional[torch.Tensor],
):
    bs = self._get_batch_size(points, boxes, masks)
    sparse_embeddings = torch.empty((bs, 0, self.embed_dim), device=self._get_device())
    if points is not None:
        coords, labels = points
        point_embeddings = self._embed_points(coords, labels, pad=(boxes is None))
        sparse_embeddings = torch.cat([sparse_embeddings, point_embeddings], dim=1)
    if boxes is not None:
        box_embeddings = self._embed_boxes(boxes)
        sparse_embeddings = torch.cat([sparse_embeddings, box_embeddings], dim=1)

    if masks is not None:
        dense_embeddings = self._embed_masks(masks)
    else:
        dense_embeddings = self.no_mask_embed.weight.reshape(1, -1, 1, 1).expand(
            bs, -1, self.image_embedding_size[0], self.image_embedding_size[1]
        )

    return sparse_embeddings, dense_embeddings
```

Figure 16.6: Forward pass of the prompt encoder

The sample inputs and outputs look like so:


```

Module Name: PromptEncoder
Input Kwargs:
  points -
    0 - tensor[64, 1, 2] f64 n=128 (1Kb) xE[10.672, 672.328] μ=186.750 σ=209.371 cuda:0 - ID:#5b3b2d-
    1 - tensor[64, 1] i32 xE[1, 1] μ=1.000 σ=0.0 cuda:0 - ID:#08f4ad-
  boxes - NoneType
  masks - NoneType
Input Args:
Outputs:
  sparse_embeddings - tensor[64, 2, 256] n=32768 (0.1Mb) xE[-1.352, 1.358] μ=-0.006 σ=0.530 cuda:0 - ID:#ca373f-
  dense_embeddings - tensor[64, 256, 64, 64] n=67188864 (0.2Gb) xE[-0.218, 0.138] μ=-0.001 σ=0.031 grad cuda:0 - ID:#3c6848-

```

Figure 16.7: Sample inputs and outputs of the prompt encoder

In the above example, we have sent 64 points to the encoder (64 x and y coordinates, hence the shape of [64,2]) and obtained sparse and dense embeddings. In the case of a text prompt, it is passed through CLIP embeddings to get the embedding corresponding to the text when passed through the prompt encoder. In the case of points/bounding boxes, they are represented by positional embeddings corresponding to the points.

4. The embeddings are then passed through a mask decoder that calculates the attention between prompt encoding and the image encoder and then outputs a set of masks.

The mask decoder architecture is as follows:

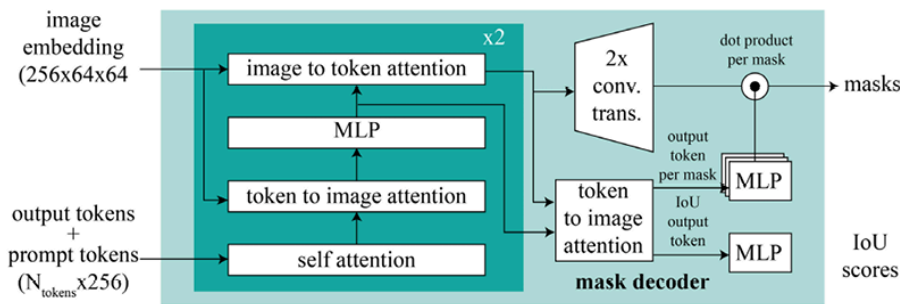


Figure 16.8: Mask decoder architecture

Here's a step-by-step explanation of the components and what they are doing:

1. **Image embedding:** The process begins with an image embedding, which is a representation of the input image transformed into a set of feature vectors (output from the vision transformer where each feature vector corresponds to a different patch).
2. **Output tokens + prompt tokens:** A prompt (any point or bounding box/text) is represented as an embedding. Output tokens are similar to

the CLS token in transformers. They are learnable embeddings that contain information for an effective segmentation exercise (where we output three possible masks corresponding to a prompt).

3. **Attention blocks:** These blocks are similar to the blocks in a transformer decoder block where we have self-attention (within the decoder output) and cross attention between the encoder and decoder.
 1. **Image to token attention:** This block helps the model to focus on specific features within the image embedding that are relevant to the tokens. It's a form of cross-attention where image features inform the processing of the tokens.
 2. **Token to image attention:** This does the reverse, allowing the tokens to influence the processing of the image features.
 3. **Self-attention:** This mechanism allows the tokens to interact with each other, helping the model to integrate information across the different tokens.
4. **Multilayer perceptron (MLP):** This is a neural network layer that processes the features from the attention mechanisms to transform and combine information further.
5. **Iterations:** The attention blocks and MLPs are stacked in layers (as indicated by x2), allowing the model to refine its understanding of the image and prompt with each iteration.
6. **2x convolutional transpose:** This is an up-sampling operation that increases the spatial resolution of the feature maps. It's also known as deconvolution. This operation is used to go from a lower-resolution embedding back to the higher-resolution space of the original image, which is necessary for creating detailed masks.
7. **Dot product per mask:** This step involves computing the dot product between the refined features and each potential mask. It's a way of scoring how well each feature corresponds to each mask, effectively aligning the feature vectors with the predicted masks.
8. **Masks:** The result of the dot product per mask is used to generate the final segmentation masks. Each mask corresponds to a particular region or object in the image as defined by the prompt.
9. **Intersection over union (IoU) scores:** Alongside the mask generation, the model also outputs IoU scores. IoU measures the overlap between the predicted segmentation mask and the ground truth mask.

The mask decoder is essentially responsible for taking the combined information from the image and the prompts (points, boxes, text, etc.) and decoding it into a set of segmentation masks that correspond to the objects or features in the image as specified by the prompts. It uses attention mechanisms to focus on relevant features, MLPs to process and combine information, and transposed convolutions to map the lower-resolution

embeddings back to the image space. The final output is a set of masks, each with a corresponding IoU score indicating the quality of the segmentation.

Implementing SAM

Imagine a scenario where you task an annotator with annotating an image as quickly as possible, with only one click (point prompt on top of the object of interest) as input.

In this section, let us go ahead and use the following code to leverage SAM on a sample image and learn how SAM helps fast segmentation given a point prompt:



The below code is available in the `SAM.ipynb` file in the `Chapter16` folder in the GitHub repo at <https://bit.ly/mcvp-2e>. Do be sure to run the code from the notebook and refer to the following explanations to understand the different steps.

1. Clone the GitHub repository and install the required packages:

```
!git clone https://github.com/facebookresearch/segment-anything.git
%cd segment-anything
!pip install -e .
```

2. Download the pre-trained vision transformer model:

```
!wget https://dl.fbaipublicfiles.com/segment_anything/sam_vit_h_4b8939.pth
```

3. Load the SAM model and create the predictor instance:

```
from segment_anything import SamAutomaticMaskGenerator, \
    sam_model_registry, SamPredictor
import cv2
sam = sam_model_registry["vit_h"]\
    (checkpoint="sam_vit_h_4b8939.pth").to('cuda')
mask_generator = SamAutomaticMaskGenerator(sam)
predictor = SamPredictor(sam)
```

The predictor will be responsible for the necessary preprocessing of the points/masks/boxes followed by the model prediction followed by the nec-

essary post-processing.

4. Load the image on which we want to perform prediction:

```
image = cv2.imread('/content/segment-anything/notebooks/images/truck.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
predictor.set_image(image)
```

5. Provide the point/points/box/prompt for which we want to extract the corresponding mask:

```
import numpy as np
input_point = np.array([[500, 375]])
input_label = np.array([1])
```

6. Extract the masks that correspond to the provided point:

```
masks, scores, logits = predictor.predict(
    point_coords=input_point,
    point_labels=input_label,
    multimask_output=True,
)
```



Figure 16.9: Masks obtained by providing a point prompt

Based on the preceding output, we can see that we are able to generate masks corresponding to the point that was provided as input. This process can be extended to:

- A box as input
- Multiple points and boxes as input
- A text prompt as input

We have provided examples for all of the above in the associated GitHub repository.

Now that we have learned about extracting masks given a prompt, let us now learn about extracting all the possible masks from an image:

1. To extract all the masks, follow steps 1-3 in the previous list of steps in this section.
2. Load the image:

```
import cv2
image = cv2.imread('/content/segment-anything/notebooks/images/truck.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

3. Generate the masks:

```
masks = mask_generator.generate(image)
```

4. Define the function to visualize masks overlaid on top of the original image:

```
import numpy as np
import torch
import matplotlib.pyplot as plt
import cv2
def show_anns(anns):
    if len(anns) == 0:
        return
    sorted_anns = sorted(anns, key=(lambda x: x['area']), reverse=True)
    ax = plt.gca()
    ax.set_autoscale_on(False)
    img = np.ones((sorted_anns[0]['segmentation'].shape[0],
                    sorted_anns[0]['segmentation'].shape[1], 4))
    img[:, :, 3] = 0
    for ann in sorted_anns:
        m = ann['segmentation']
        color_mask = np.concatenate([np.random.random(3), [0.35]])
        img[m] = color_mask
    ax.imshow(img)
```

5. Visualize the image:

```
plt.figure(figsize=(20,20))
plt.imshow(image)
show_anns(masks)
plt.axis('off')
plt.show()
```

The output is as follows:



Figure 16.10: All the predicted masks within an image

From the preceding image, we can see that we have masks of different granular regions in the original image.

So far, we have learned about leveraging the SAM to identify masks in an image. However, it took a considerable amount of time (5-20 seconds depending on the image resolution and the objects in the image) to fetch the masks for one image, making it very difficult to use when leveraging it for real-time segmentation. In the next section, we will learn about FastSAM, which helps with the real-time generation of masks.

How FastSAM works

The SAM takes input prompts to calculate corresponding masks. If an input prompt is provided, it passes the masks through a second block where prompt encodings are done and if input prompts are not provided, the SAM will calculate all the possible masks. All this leverages transformers for encoding images and also attention calculations while decoding. This takes a considerable time (~10 seconds) to process an input image and the prompts. How could we reduce the time it takes to generate predictions? FastSAM helps in achieving that.

FastSAM breaks down the SAM into two tasks:

1. Calculating the masks of instances
2. Associating the prompt with one of the instances

In addition, FastSAM leverages a CNN backbone, which further reduces the computation complexity. FastSAM is trained on only 2% of the data points present in the SAM 1B dataset and its architecture is as follows:

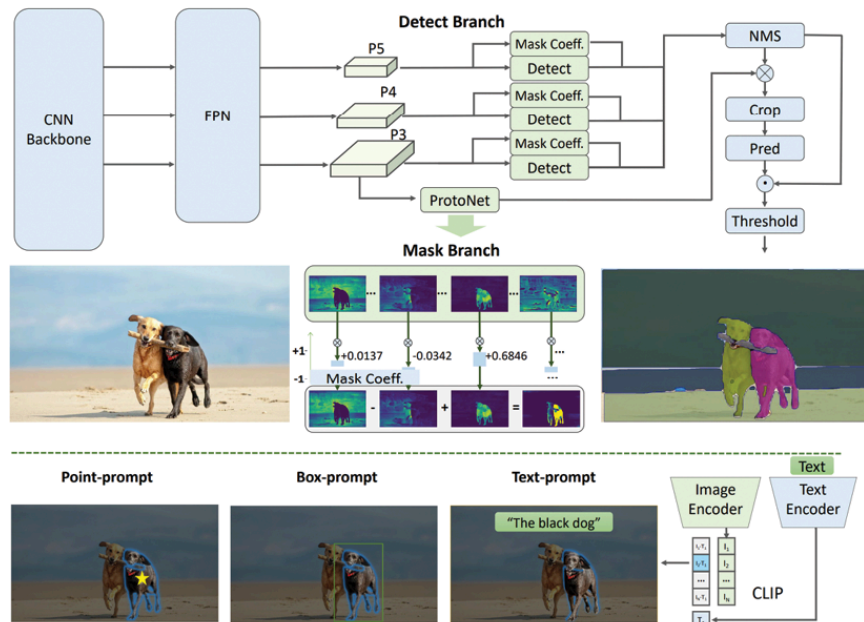


Figure 16.11: FastSAM workflow (source:

<https://arxiv.org/pdf/2306.12156>)

FastSAM involves two steps:

1. All-instance segmentation
2. Prompt-guided selection

Let's look at these steps in some detail.

All-instance segmentation

In Figure 16.11 (in the top half of the workflow), we pass an input image through a CNN backbone (like the ResNet101 architecture). We further pass the output through a feature pyramid network that not only extracts features from a given image but also ensures that diverse size features are captured. The output of FPN has two different branches – detection and segmentation.

The detection branch outputs the category and bounding box, while the segmentation branch outputs k prototypes (defaulted to 32 in FastSAM) along with k mask coefficients (for more details on prototypes and mask coefficients, refer to the YOLACT++ paper here:

<https://arxiv.org/pdf/1912.06218.pdf>). The segmentation and detection tasks are computed in parallel. The segmentation branch inputs a high-resolution feature map, preserves spatial details, and also contains semantic information.

Prompt-guided selection

Prompt-guided selection then becomes easier as the point that falls within a mask then highlights the instance/mask (in the case of a point prompt) and the box that has the highest IoU determines the instance in the case of a box prompt.

Further, in the case of text embedding, we leverage the CLIP model to calculate the text embeddings and compare those with the embeddings of each instance in the image to identify the mask that is most likely to represent the text.

Now that we understand how FastSAM works at a high level, let us go ahead and implement it.

Implementing FastSAM

To implement FastSam, use the following code:



This code is available in the `FastSAM.ipynb` file in the `Chapter16` folder in the GitHub repo at <https://bit.ly/mcyp-2e>. Do be sure to run the code from the notebook and refer to the following explanations to understand the different steps.

1. Clone the Git repository associated with FastSAM:

```
!git clone https://github.com/CASIA-IVA-Lab/FastSAM.git
```

2. Download the pre-trained model:

```
!wget https://huggingface.co/spaces/An-619/FastSAM/resolve/main/weights/FastSAM.pt
```

3. Install the required packages and OpenAI clip:

```
!pip install -r FastSAM/requirements.txt
```

```
!pip install git+https://github.com/openai/CLIP.git
```

4. Change the directory:

```
%cd FastSAM
```

5. Import the required packages:

```
import matplotlib.pyplot as plt
import cv2
from fastsam import FastSAM, FastSAMPrompt
```

6. Instantiate the model and the image on which we want to work:

```
model = FastSAM('/content/FastSAM.pt')
IMAGE_PATH = '/content/FastSAM/images/cat.jpg'
DEVICE = 'cuda'
```

7. Pass the image through the model:

```
everything_results = model(IMAGE_PATH, device=DEVICE,
                           retina_masks=True, imgsz=1024,
                           conf=0.4, iou=0.9)
prompt_process = FastSAMPrompt(IMAGE_PATH, everything_results, \
                                device=DEVICE)

# everything prompt
ann = prompt_process.everything_prompt()
```

The preceding code generates all the possible masks that are detected in the image. Note that the execution finishes in 0.5 seconds (which is ~20X faster than SAM).

8. Specify the prompt using which we want to obtain masks:

```
ann = prompt_process.text_prompt(text='a photo of a cat')
```

9. Write the processed image to disk:

```
prompt_process.plot(annotations=ann, output_path='./output/cat.jpg')
```

The preceding code saves the picture with the corresponding mask of a cat.

So far, we have learned about leveraging SAM to get masks corresponding to an image and FastSAM to speed up the generation of predictions. However, in a real-world scenario, we might want to keep track of a given instance/object over time across all frames present in a video. The paper *Segment & Track Anything* (<https://arxiv.org/pdf/2305.06558.pdf>) gives details on how to do that.



The code to track anything is provided as `SAMTrack.ipynb` in the associated GitHub repository.

Further, ImageBind is another foundation model that binds multiple modalities – image, text, audio, video, heatmap, and depth – into one dimension and thus could provide an opportunity to translate across modalities. For details regarding ImageBind, do go through the associated paper and GitHub repository (<https://github.com/facebookresearch/ImageBind>).

So far, we’ve learned about zero-shot recognition or zero-shot segmentation given an image. In the next section, we will learn about diffusion models, which will help with the zero-shot **generation** of an image.

Introducing diffusion models

In the previous GAN chapters, we learned about generating images from noise; we also learned about generating images from conditional input like the class of images that should be generated. However, in that scenario, we were able to get the image of a face straight away from random noise. This is a step change. What if we could generate an image from random noise in a more incremental way? For example, what if we could gradually generate the contour corresponding to the image initially and then slowly get the finer details of the image from the contours over increasing epochs? Further, what if we could generate an image from text input? Diffusion models come in handy in such a scenario.

A diffusion model mimics the scenario of a diffusion process, which refers to the gradual spread or dispersion of a quantity (in this case, pixel values in an image) over time.

How diffusion models work

Imagine a scenario where you have a set of images. In the forward process, we add a small amount of noise over increasing time steps – i.e.,

in the first iteration, the amount of noise is very low, however, in the 100th iteration, the amount of noise is very high.

In the reverse process, we take the noisy image (at the 100th timestep) as the input and predict the amount of noise present in the image. Next, we remove the predicted noise, add a smaller amount of noise (to maintain stability; more on this in the next section and the associated GitHub repository) and predict the noise present in the 99th timestep and will repeat the above sequence of steps until we reach the first timestep.

A diffusion model is a typical U-Net model (modified with attention and resnet modules) that works as follows:

1. The input image is a noisy image with varying noise depending on the timestep (t)
2. Pass the image along with the time embeddings (corresponding to the timestep) through a few convolution and pooling steps to get an encoder vector
3. Pass the encoder vector through up-convolutions and add skip connections from input just like we did in the segmentation exercise in the U-Net architecture in *Chapter 9*
4. Predict the noise present in the input image
5. Subtract the predicted noise from the input image
6. Repeat from step 1 for timestep t-1 with a slight amount of noise added back to the image obtained in step 5

In the preceding process, there are a couple of aspects to consider:

- The amount of noise to add in each time step – the noise curve
- The amount of noise to be added back after subtracting noise at a given time step

In our exercise, we will follow the pattern shown in this graph, where the x axis is the timestep and the y axis is the amount of preservation of the original (input) image:

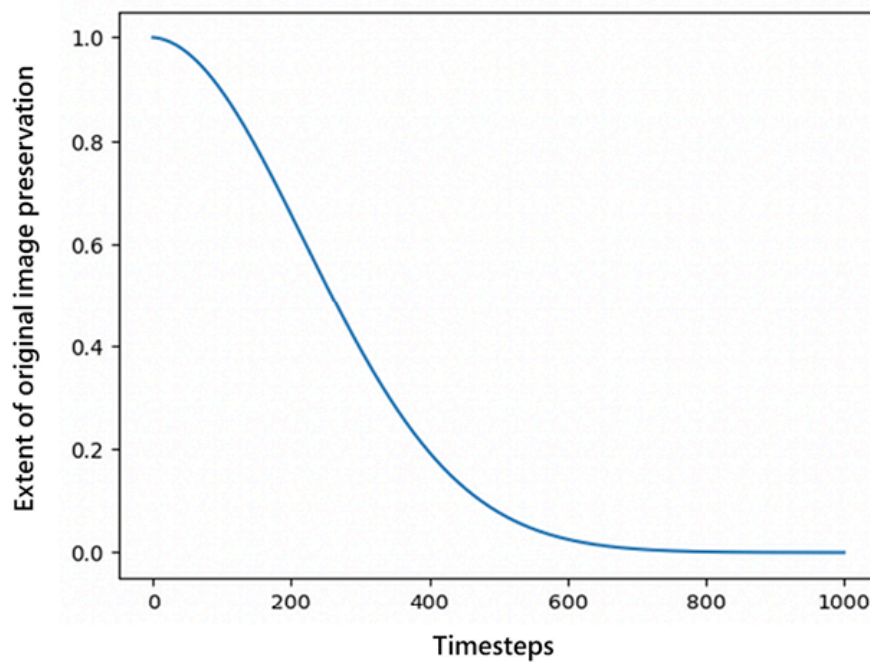


Figure 16.12: Extent of noise added to an image over increasing timesteps

Initially, we would not modify the input image a lot. However, over increasing timesteps, we add considerable noise to the image.

Such a pattern of noise addition is more helpful than the linear addition of noise as this would help the model to take its time in picking the contours and details of the image, whereas the linear addition of noise would force the model to start learning right from the very first timestep.

Now that we understand how diffusion models work at a high level, let us understand the architecture details of diffusion models in the next section.

Diffusion model architecture

To understand the architecture, we will build a diffusion model to generate MNIST digits. This requires inputs that are 28x28x1 in shape and generates outputs that are of the same shape. But first, let's look at the architecture in detail.

A diffusion model is a UNet model with the following layers/blocks:

- Convolution
- Timestep embedding
- Attention blocks
- ResNet blocks

- Downsampling



A quick overview of the architecture is provided in the `unet_components_from_scratch.ipynb` notebook on GitHub at <https://bit.ly/mcyp-2e>.

The architecture of the diffusion U-Net is as follows:

```

=====
Layer (type:depth-idx)                               Param #
=====
UNet2DModel                                           --
├─Conv2d: 1-1                                         320
├─Timesteps: 1-2                                       --
├─TimestepEmbedding: 1-3                             --
│   └─Linear: 2-1                                     4,224
│   └─SiLU: 2-2                                       --
│   └─Linear: 2-3                                     16,512
├─ModuleList: 1-4
│   ├──DownBlock2D: 2-4                               32,000
│   ├──AttnDownBlock2D: 2-5                           119,680
│   ├──AttnDownBlock2D: 2-6                           460,544
│   └──AttnDownBlock2D: 2-7                           1,215,744
├─ModuleList: 1-5
│   ├──AttnUpBlock2D: 2-8                             4,661,248
│   ├──AttnUpBlock2D: 2-9                             1,347,840
│   ├──AttnUpBlock2D: 2-10                             346,240
│   └──UpBlock2D: 2-11                                78,528
├─UNetMidBlock2D: 1-6
│   ├──ModuleList: 2-12                             263,680
│   └──ModuleList: 2-13                             2,428,416
├─GroupNorm: 1-7                                     64
├─SiLU: 1-8                                           --
└─Conv2d: 1-9                                         289
=====
Total params: 10,975,329
Trainable params: 10,975,329
Non-trainable params: 0
=====

```

Figure 16.13: Summary of Diffusion U-Net architecture

Let us understand the different modules/blocks present in the architecture:

- The convolution module – `Conv_in` – takes the original image and increases the number of channels present in the image.
- The `time_embedding` module takes the current timestep and converts it into an embedding.
- A `down_block` consists of the following modules:

- ResNet: A **ResNet** block consists of the following modules:
 - Group normalization: One of the limitations of training models to generate images with high resolution is that on a standard GPU, one cannot fit many images in one batch. This results in batch normalization being limited to a smaller number, resulting in less stable training.
 - Group normalization comes in handy in such a scenario, as it operates on channels within one input and not on a batch of inputs. In essence, group normalization does the normalization (a mean of zero and a variance of one) across channels within an image. This way, we ensure that a group of channels has a standard data distribution and thus ensures more stable training.
 - Convolution: Once we perform group normalization, we pass the output through a convolution layer.
 - Time embedding projection: Next, we add the time embedding by projecting time embedding to a dimension that is consistent with the output of group normalization.
 - Non-linearity: The non-linearity used within this module is **Sigmoid Linear Unit (SiLU)**, which is calculated as follows:

$$SiLU(x) = x * \frac{1}{1 + e^{-x}}$$
- **Attention**: The output is then passed to the **attention** block, which does the following operations:
 - We first perform group normalization and increase the number of groups present in the input.
 - Next, we perform attention (just like we did using key, query, and value matrices in the previous chapter).
 - Finally, we pass the output through a linear layer.
- Downsampling: In the **downsampling** block, we take an input and reduce the dimensions to half (stride = 2).

The exact reverse of the above steps happens in the **upsampling** blocks. Further, skip connections from the input are added during upsampling. We will cover each and every layer in greater depth in the next section – *Understanding Stable Diffusion*.

Now that we understand the architecture of diffusion models, let us go ahead and build a diffusion model on the MNIST dataset in the following section.

Implementing a diffusion model from scratch

To implement a diffusion model, we will leverage the MNIST dataset and perform the following steps:



The following code is available in the `Diffusion_PyTorch.ipynb` file in the `Chapter16` folder in the GitHub repo at <https://bit.ly/mcvp-2e>. Do be sure to run the code from the notebook and refer to the following explanations to understand the different steps.

1. Install the required libraries and load the libraries:

```
%pip install -q diffusers torch-snippets
from torch_snippets import *
from diffusers import DDPMsScheduler, UNet2DModel
from torch.utils.data import Subset
import torch
import torch.nn as nn
from torch.optim.lr_scheduler import CosineAnnealingLR
device = 'cuda' # torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f'Using device: {device}')
```

2. Load the dataset:

```
transform = torchvision.transforms.Compose([
    torchvision.transforms.Resize(32),
    torchvision.transforms.ToTensor()
])
dataset = torchvision.datasets.MNIST(root="mnist/", train=True, \
    download=True, transform=transform)
```

3. Define the batch size and `train_dataloader`:

```
batch_size = 128
train_dataloader = DataLoader(dataset,
    batch_size=batch_size,
    shuffle=True)
```

4. Define the model architecture, where we will leverage the `UNet2DModel` from the `diffusers` library. This consists of all the different blocks in the architecture that we mentioned in the previous section:

```

net = UNet2DModel(
    sample_size=28, # the target image resolution
    in_channels=1, # the number of input channels, 3 for RGB images
    out_channels=1, # the number of output channels
    layers_per_block=1, # how many ResNet layers to use per UNet block
    block_out_channels=(32, 64, 128, 256), # Roughly matching our basic unet exan
    down_block_types=(
        "DownBlock2D", # a regular ResNet downsampling block
        "AttnDownBlock2D", # a ResNet downsampling block with spatial self-attenti
        "AttnDownBlock2D",
        "AttnDownBlock2D",
    ),
    up_block_types=(
        "AttnUpBlock2D",
        "AttnUpBlock2D",
        "AttnUpBlock2D", # a ResNet upsampling block with spatial self-attention
        "UpBlock2D", # a regular ResNet upsampling block
    ),
)
_ = net.to(device)

```

5. Define the noise scheduler:

```
noise_scheduler = DDPMsScheduler(num_train_timesteps=1000)
```

The `DDPMsScheduler` (**DDPM** stands for **Denoising Diffusion Probabilistic Models**) is a component that manages the scheduling of this noise addition and reversal process. It determines at what rate and in what manner the noise should be added and then reversed. The scheduler typically controls aspects such as:

- The number of diffusion steps
- The variance of noise added at each step
- How the variance changes over the steps during the reverse process

6. Define a function that takes an input image along with the corresponding timestep and corrupts the image:

```

def corrupt(xb, timesteps=None):
    if timesteps is None:
        timesteps = torch.randint(0, 999, (len(xb),)).long().to(device)
    noise = torch.randn_like(xb)
    noisy_xb = noise_scheduler.add_noise(xb, noise, timesteps)
    return noisy_xb, timesteps

```

7. Define the model training configuration:

```

n_epochs = 50
report = Report(n_epochs)
loss_fn = nn.MSELoss()
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
scheduler = CosineAnnealingLR(opt, T_max=len(train_dataloader), \
                               verbose=False)

```

In the preceding code, `CosineAnnealingLR` adjusts the learning rate following a cosine annealing schedule. This means the learning rate keeps decreasing from an initial high value to a minimum value, and then increases again. This could potentially result in avoiding local minima.

The parameters of the scheduler are:

- `opt` : The optimizer for which the scheduler is adjusting the learning rate.
- `T_max` : The number of iterations after which the learning rate will reset. In your code, it's set to the length of `train_dataloader`, meaning the learning rate will complete a cosine cycle after each epoch.

8. Train the model:

```

for epoch in range(n_epochs):
    n = len(train_dataloader)
    for bx, (x, y) in enumerate(train_dataloader):
        x = x.to(device) # Data on the GPU
        noisy_x, timesteps = corrupt(x) # Create our noisy x
        pred = net(noisy_x, timesteps).sample
        loss = loss_fn(pred, x) # How close is the output to the true 'clean' x?
        opt.zero_grad()
        loss.backward()
        opt.step()
        scheduler.step()
        report.record(epoch + ((bx + 1) / n), loss=loss.item(), end='\r')
    report.report_avgs(epoch + 1)

```

The plot of loss value is:

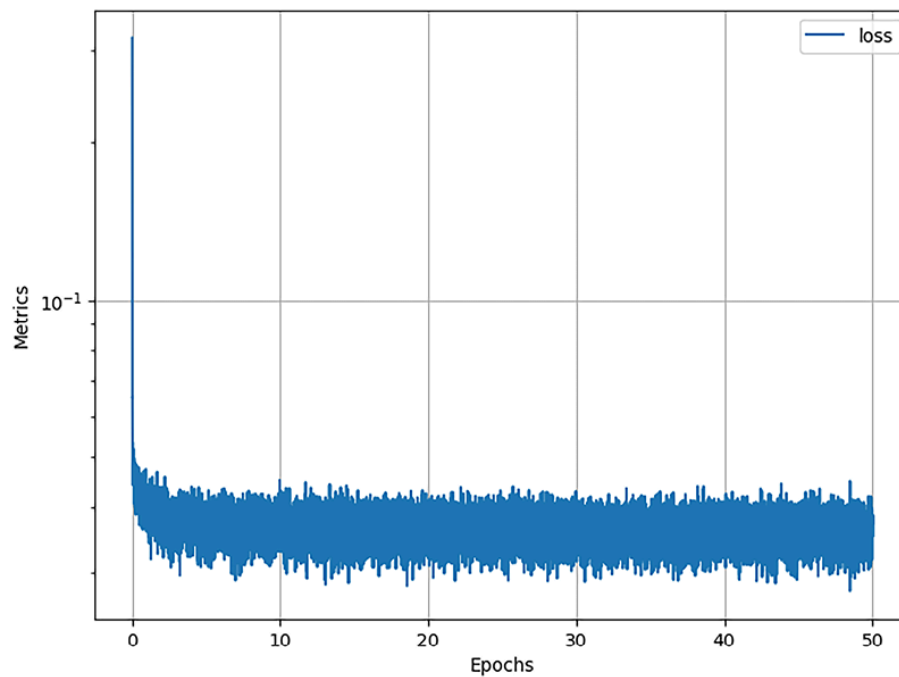


Figure 16.14: Loss value over increasing epochs

9. Let's plot a few data points that are generated in this process:

```
net.cpu()
noise = torch.randn(5,1,32,32).to(net.device)
progress = [noise[:,0]]
for ts in np.logspace(np.log10(999), 0.1, 100):
    ts = torch.Tensor([ts]).long().to(net.device)
    noise = net(noise, ts).sample.detach().cpu()
    noise, _ = corrupt(noise, ts)
    progress.append(noise[:,0])
print(len(progress))
_n = 10
subplots(torch.stack(progress[::_n]).permute(1, 0, 2, 3).reshape(-1, 32, \
                                                                    32), nc=11, sz=(10,4))
```

The preceding code results in:

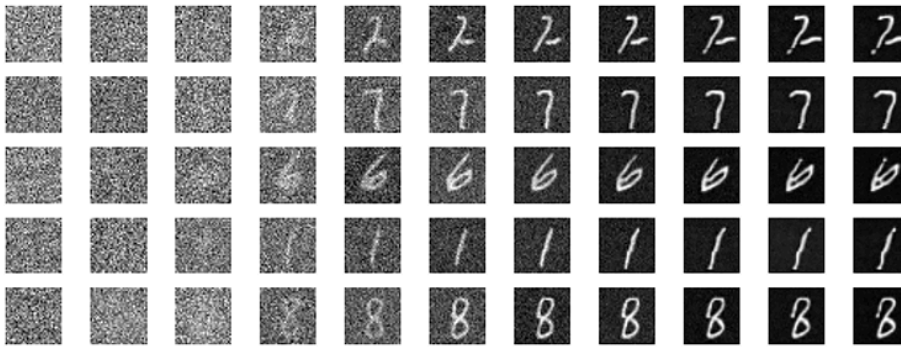


Figure 16.15: Image generated over increasing time steps

From the above, we can see that we are able to generate MNIST digits from random noise.

However, while the current model is able to generate images, we are unable to specify the label that is of interest to us. In the next section, we will learn about how to add additional context (like text prompts) to generate images conditionally.

Conditional image generation

To train a diffusion model with conditional inputs, we modify the following:

1. Extend the UNet so that it accepts additional input channels. This way, the prompt is appended to the original channels of input.
2. Pass the label through an embedding layer so that we convert it to an embedding.
3. Modify the image corrupt function to concatenate the embeddings of labels along with input images.

Once the above changes are done, the rest of the training code remains the same. Let us go ahead and code conditional image generation:



The below code is available in the `Conditional_Diffuser_training.ipynb` file in the `Chapter16` folder in the GitHub repo at <https://bit.ly/mcvp-2e>. Do be sure to run the code from the notebook and refer to the following explanations to understand the different steps.

1. Steps 1-3 remain the same as in the previous section.

2. Next, define the embedding layer:

```
class EmbeddingLayer(nn.Module):
    def __init__(self, num_embeddings, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(num_embeddings,
                                      embedding_dim)

    def forward(self, labels):
        return self.embedding(labels)

embedding_layer = EmbeddingLayer(num_embeddings=10, embedding_dim=32).to(device)
```

In the preceding code, we are creating an embedding layer that takes any of the 10 possible labels and converts it into an embedding of dimension 32.

3. Extend the `UNet` class so that it accepts 32 additional input channels:

```
class ConditionalUNet2DModel(UNet2DModel):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.in_channels += 32 # Adjust for embedding dimension
```

4. Instantiate the UNet2D model object:

```
net = ConditionalUNet2DModel(
    sample_size=28,
    in_channels=1 + 32, # 1 for original channel, 32 for embedding
    out_channels=1,
    layers_per_block=1,
    block_out_channels=(32, 64, 128, 256),
    down_block_types=("DownBlock2D", "AttnDownBlock2D",
                     "AttnDownBlock2D", "AttnDownBlock2D"),
    up_block_types=("AttnUpBlock2D", "AttnUpBlock2D",
                   "AttnUpBlock2D", "UpBlock2D"),).to(device)
```

5. Define the image corrupt function where the image corruption is done in a manner similar to what we saw in the *How diffusion models work* section:

```
def corrupt_with_embedded_labels(xb, labels, timesteps=None):
    if timesteps is None:
        timesteps = torch.randint(0, 999,
                                  (len(xb),)).long().to(device)
    noise = torch.randn_like(xb)
    noisy_xb = noise_scheduler.add_noise(xb, noise,
```

```

                                timesteps)
labels_embedded=embedding_layer(labels).unsqueeze(1).unsqueeze(-1)
labels_embedded = labels_embedded.expand(-1, -1,
                                xb.shape[2], xb.shape[3])
return torch.cat([noisy_xb, labels_embedded], dim=1), timesteps

```

In the preceding code, we pass the labels through `embedding_layer` to fetch their corresponding embeddings. Finally, we concatenate the noisy image and the label embedding.

6. Train the model just like we did in the previous section, in step 8.

7. Perform inference. To do this, we initialize 10 images with zero pixel values and specify that the timestamp is 999:

```

xb = torch.zeros(10, 1, 32, 32)
timesteps = torch.randint(999, 1000, (len(xb),)).long().to(device)

```

8. Next, we add noise to the initialized images using `noise_scheduler`:

```

noise = torch.randn_like(xb)
noisy_xb = noise_scheduler.add_noise(xb, noise, timesteps).to(device)

```

9. Define the labels we want to generate (we want to generate one image corresponding to each label):

```

labels = torch.Tensor([0,1,2,3,4,5,6,7,8,9]).long().to(device)

```

10. Fetch embeddings corresponding to the labels:

```

labels_embedded=embedding_layer(labels).unsqueeze(-1).unsqueeze(-1)
labels_embedded = labels_embedded.expand(-1, -1,
                                xb.shape[2], xb.shape[3]).to(device)

```

11. Concatenate the noisy image and the label embeddings:

```

noisy_x = torch.cat([noisy_xb, labels_embedded], dim=1)

```

12. Make predictions using the trained model:

```

pred = net(noisy_x, timesteps).sample.permute(0,2,3,1).reshape(-1, 32, 32)

```


13. Visualize the generated images:

```
subplots(pred.detach().cpu().numpy())
```



Figure 16.16: Generated images

From the preceding output, we see that we can conditionally generate images by specifying their labels.

Now that we have learned about generating images using a diffusion model from scratch, we will learn about leveraging Stable Diffusion to generate images given a text prompt.

Understanding Stable Diffusion

So far, we've learned how diffusion models work. Stable Diffusion improves upon the UNet2D model by first leveraging VAE to encode an image to a lower dimension and then performing training on the down-scaled/latent space. Once the model training is done, we use a VAE decoder to get a high-resolution image. This way, training is faster as the model learns features from the latent space than from the pixel values.

The architecture of Stable Diffusion is as follows:

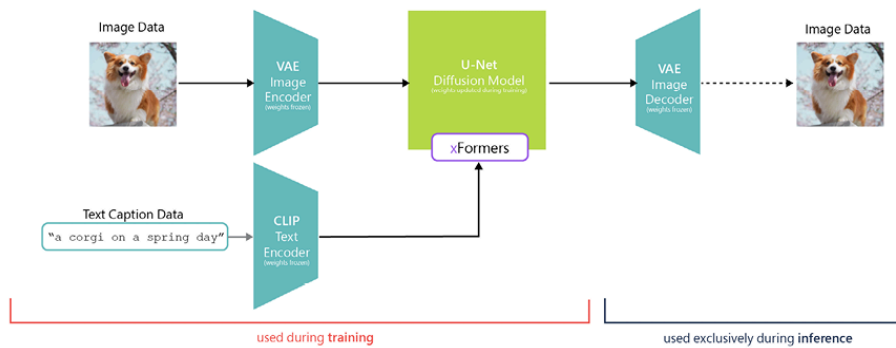


Figure 16.17: Stable Diffusion overview

The VAE encoder is a standard auto-encoder that takes an input image of shape 768x768 and returns a 96x96 image. The VAE decoder takes a 96x96 image and upscales it to 768x768.

The pre-trained Stable Diffusion U-Net model architecture is:

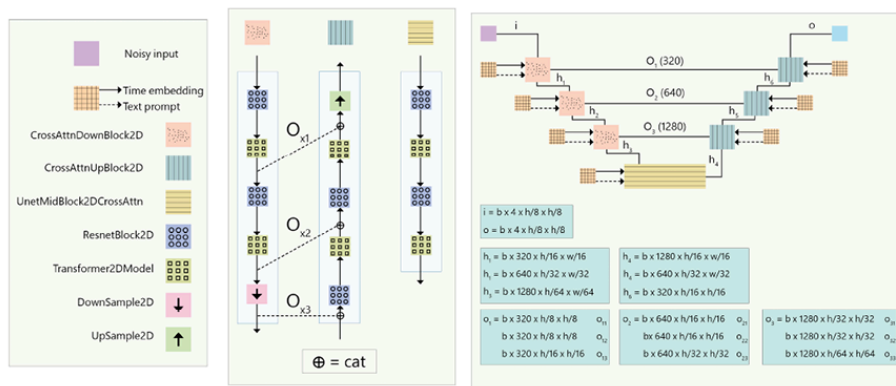


Figure 16.18: Pre-trained Stable Diffusion U-Net model architecture

In the preceding diagram, noisy input represents the output obtained from the VAE encoder. Text prompt represents the CLIP embeddings of text.

Building blocks of the Stable Diffusion model

The UNet2D architecture is central to the functionality of Stable Diffusion, an in-depth understanding of it essential for mastering the Stable Diffusion landscape. Given its complexity, characterized by numerous connections and data flows between layers, it is crucial to comprehensively understand each key layer. To achieve this, we will examine each layer through three critical components:

1. Inputs to the Layer: What data or tensors are fed into the layer?
 2. Transformation of Inputs: How does the layer process or transform these inputs?
 3. Final Outputs: What is the resulting output after the transformation?
- By delving into these aspects, we aim to provide a clear picture of how tensors flow through the model, thereby enabling you to master Stable Diffusion thoroughly.

Let us understand what is under the hood of the UNet2D model by downloading a pretrained Stable Diffusion model and calling its summary, like so:

```
from diffusers import StableDiffusionPipeline
from torchinfo import summary
model_id = "stabilityai/stable-diffusion-2"
pipe = StableDiffusionPipeline.from_pretrained(model_id)
summary(pipe.unet, depth=4)
```

We get the following:

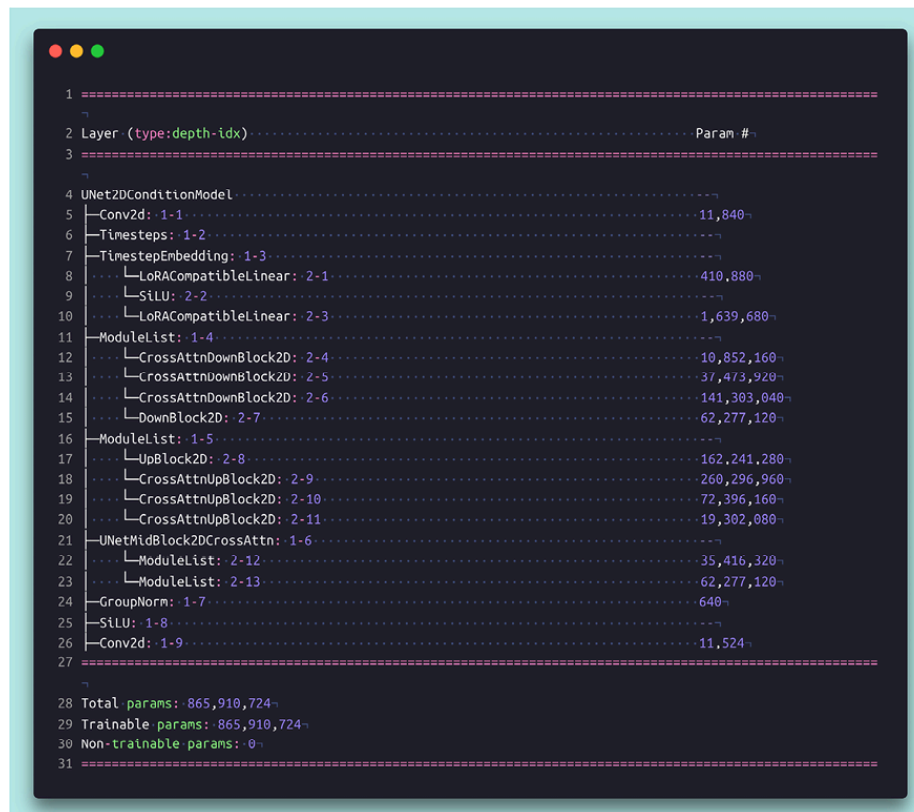


Figure 16.19: UNet2D model architecture

Let's try to uncover each piece in the pipeline for an in-depth understanding of the theory behind such a successful network. We have the following high-level blocks:

- CrossAttnDownBlock2D
- CrossAttnUpBlock2D
- UNetMidBlock2DCrossAttn
- DownBlock2D
- UpBlock2D

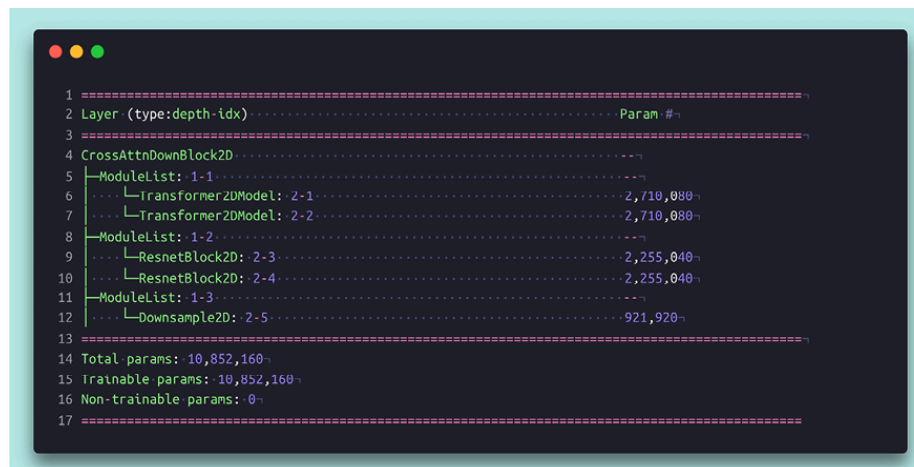
Let's look at each of these blocks in detail.

CrossAttnDownBlock2D

Running `pipe.unet.down_blocks` gives us the four down-sampling modules (i.e., the left part of UNet in *Figure 16.18*). The first three are `CrossAttnDownBlock2D` and the last one is `DownBlock2D`. On checking the summary for any of the `CrossAttnDownBlock2D`, we get:

```
Summary(pipe.unet.down_blocks[0], depth=2)
```

The preceding code results in:



```

1 =====
2 Layer (type:depth-idx) ..... Param #
3 =====
4 CrossAttnDownBlock2D .....
5 | ModuleList: 1-1 .....
6 | | Transformer2DModel: 2-1 ..... 2,710,080
7 | | Transformer2DModel: 2-2 ..... 2,710,080
8 | | ModuleList: 1-2 .....
9 | | | ResnetBlock2D: 2-3 ..... 2,255,040
10 | | | ResnetBlock2D: 2-4 ..... 2,255,040
11 | | ModuleList: 1-3 .....
12 | | | Downsample2D: 2-5 ..... 921,920
13 =====
14 Total params: 10,852,160
15 Trainable params: 10,852,160
16 Non-trainable params: 0
17 =====

```

Figure 16.20: Summary of the first down block of the unet

As you can see in *Figure 16.20*, a module is made of 3 types of blocks – `Transformer2DModel`, `ResnetBlock2D`, and `Downsample2D`.

Inspecting the `forward` method of the `CrossAttnDownBlock2D` class in the github.com/huggingface/diffusers GitHub repo at

`diffusers/models/unet_2d_blocks.py`, we see that the model works with three inputs:

```

1 hidden_states -- tensor[2, 320, 96, 96] n=5898240 (22Mb) xE[-3.921, -3.216] μ=0.004 σ=0.351 cuda:0
2 temb -- tensor[2, 1280] n=2560 (10Kb) xE[-2.014, 5.339] μ=-0.003 σ=0.256 cuda:0
3 encoder_hidden_states -- tensor[2, 77, 1024] n=157696 (0.6Mb) xE[-6.571, 13.023] μ=-0.169 σ=1.032 cuda:0

```

Figure 16.21: Inputs for `CrossAttnDownBlock2D`

As shown in the preceding screenshot, these inputs are:

- `hidden_states`, which is the embeddings corresponding to the noisy x
- `temb`, which corresponds to the embeddings of `timestep`
- `encoder_hidden_states`, which corresponds to the embeddings of the input text prompt

At a high level, the (simplified) order of computations within a `CrossAttnDownBlock2D` block (we have modified the code for better explainability) is provided in Figure 16.22.

```

1 def forward(
2     self,
3     hidden_states: torch.FloatTensor,
4     temb: Optional[torch.FloatTensor] = None,
5     encoder_hidden_states: Optional[torch.FloatTensor] = None,
6 ):
7     output_states = ()
8     for i, (resnet, attn) in enumerate(blocks):
9         hidden_states = resnet(hidden_states, temb)
10        hidden_states = attn(
11            hidden_states,
12            encoder_hidden_states=encoder_hidden_states,
13        )[0]
14        output_states = output_states + (hidden_states,)
15        if self.downsamplers is not None:
16            for downsampler in self.downsamplers:
17                hidden_states = downsampler(hidden_states, scale=lora_scale)
18            output_states = output_states + (hidden_states,)
19    return hidden_states, output_states

```

Figure 16.22: Forward method for `CrossAttnDownBlock2D`



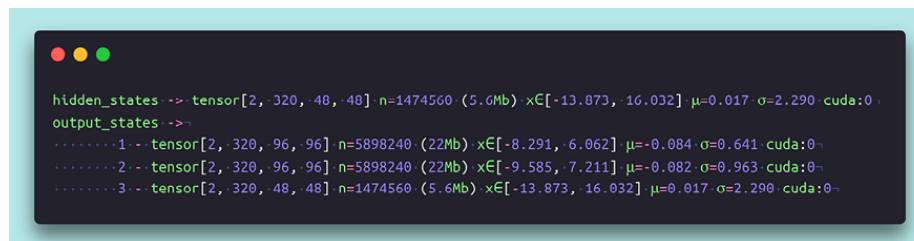
In the architecture diagram in Figure 16.20, note that `CrossAttnDownBlock2D` has ResNet and Transformer2D blocks repeated twice before performing `DownBlock2D`, and that is reflected as a `for` loop in Figure 16.22.

The key point to understand in the flow is that the `resnet` block uses `temb`, whereas the `attn` module uses `encoder_hidden_states` (which is related to the prompt).

Ultimately, `CrossAttnDownBlock2D` takes the input hidden states and passes them through its internal blocks in the following order, along with necessary `temb` or `encoder_hidden_states` tensors as inputs to `resnet` and `attn` respectively:

`resnet -> attn -> resnet -> attn -> downsampler`

Finally, the hidden states and intermediate hidden states are returned:



```
hidden_states -- tensor[2, 320, 48, 48] n=1474560 (5.6Mb) xE[-13.873, -16.032] μ=0.017 σ=2.290 cuda:0
output_states --
.....1-- tensor[2, 320, 96, 96] n=5898240 (22Mb) xE[-8.291, -6.062] μ=-0.084 σ=0.641 cuda:0
.....2-- tensor[2, 320, 96, 96] n=5898240 (22Mb) xE[-9.585, -7.211] μ=-0.082 σ=0.963 cuda:0
.....3-- tensor[2, 320, 48, 48] n=1474560 (5.6Mb) xE[-13.873, -16.032] μ=0.017 σ=2.290 cuda:0
```

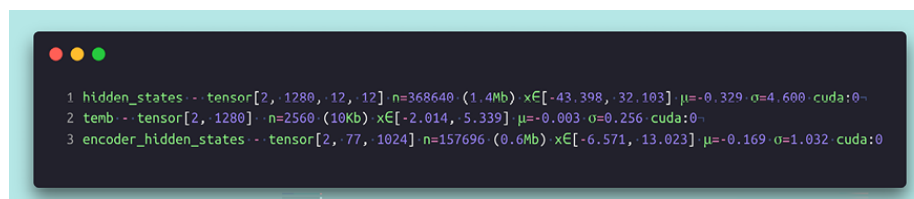
Figure 16.23: Outputs for `CrossAttnDownBlock2D`

Note that we obtain 320 channels as `DownBlock2D` gives 320 channels as output. The `hidden_states` are used as input for the next block, while output states are used for skip connections. This step is repeated 2 more times with an increasing number of channels before being fed to the middle block.

UNetMidBlock2DcrossAttn

This block is the bottleneck of the UNet, acting as the point where the important information is distilled as much as possible.

It takes the following inputs:



```
1 hidden_states -- tensor[2, 1280, 12, 12] n=368640 (1.4Mb) xE[-43.398, -32.103] μ=-0.329 σ=4.600 cuda:0
2 temb -- tensor[2, 1280] n=2560 (10Kb) xE[-2.014, -5.339] μ=-0.003 σ=0.256 cuda:0
3 encoder_hidden_states -- tensor[2, 77, 1024] n=157696 (0.6Mb) xE[-6.571, -13.023] μ=-0.169 σ=1.032 cuda:0
```

Figure 16.24: Inputs for `UNetMidBlock2DcrossAttn`

It also uses the following (simplified) forward method:

Figure 16.25: forward method (simplified) for UNetMidBlock2DCrossAttn

Note that this module has an additional `ResNet`. The output is as follows:

Figure 16.26: Output for UNetMidBlock2DCrossAttn

This output is fed as `hidden_states` to the next `CrossAttnUpBlock2D` module.

CrossAttnUpBlock2D

The summary for this module is similar to that of `CrossAttnDownBlock2D`. One might think the only difference here is in the names – just `Up` wherever `Down` is present. But the important thing to consider is the fact that these blocks additionally accept the output states (`res_hidden_states_tuple` in the following screenshot) coming from the corresponding level of `CrossAttnDown2D`:

Figure 16.27: CrossAttnUpBlock2D

The simplified forward method looks like the following:

Figure 16.28: forward method (simplified) for CrossAttnUpBlock2D

The additional input called `res_hidden_states_tuple` is a collection of three tensors (as there are three `output_states` per `down` block). Each tensor is concatenated with `hidden_states` and fed to the `resnet`.

The output is a single tensor of hidden states:

Figure 16.29: Output for CrossAttnUpBlock2D

Note that we get output of shape 96x96, which is the input image shape.

DownBlock2D, UpBlock2D

These two blocks contain only Resnet2D blocks and do not have attention modules. The activity to find the inputs, outputs, and forward of these blocks is left as an exercise for the reader. You are encouraged to go through github.com/huggingface/diffusers and search for the class definitions of `DownBlock2D` and `UpBlock2D`.

Transformer2DModel

This is a standard self-attention-based transformer encoder module that has an additional responsibility to convert 2D inputs to 1D during the input phase and back to 2D outputs at the end.

Here are the inputs:

Figure 16.30: Input of Transformer2DModel

A (simplified) `forward` method for the module is given below:

Figure 16.31: forward method (simplified) for Transformer2DModel

The input is stored as residual at first and then the `hidden_states` tensor is projected through a convolution layer (`self.proj` in in the preceding code, resulting in a shape of 2x786x96x96). The height and width dimensions are rearranged in the shape `[bs, hwx, channels]`. Once this is

passed through the transformer blocks, the input and output shapes are the same. The output of this step is reshaped and permuted to (bs x h x w x channels) and is passed through a convolution to get the original dimensions:

Figure 16.32: Output of Transformer2DModel

ResnetBlock2D

This block is nothing different from a standard resnet, except for the acceptance of a new `temb` variable. Here are the inputs:

Figure 16.33: Input for ResnetBlock2D

Here's the `forward` method:

Figure 16.34: forward method for ResnetBlock2D

Here, `self.time_emb_proj` is a linear layer making sure the channel's dimension for `temb` is the same as `hidden_states`.

The output is simply the same shape as `hidden_states`:

Figure 16.35: Output for ResnetBlock2D

We now take the 320x96x96 dimensional output and pass it through the VAE decoder to fetch the output image.

Now that we understand the working details of the Stable Diffusion model, let us go ahead and leverage it to take a text prompt and convert it to an image.

Implementing Stable Diffusion

To generate images given a text prompt, we will leverage the `diffusers` library built by the Hugging Face team. We will use the following code to do this:



This code is available in the `Conditional_Diffuser_training.ipynb` file in the `Chapter16` folder in the GitHub repo at <https://bit.ly/mcwp-2e>. Do be sure to run the code from the notebook and refer to the following explanations to understand the different steps.

1. Log in to Hugging Face and provide the auth token:

```
!huggingface-cli login
```

2. Install the required packages:

```
!pip install -q accelerate diffusers
```

3. Import the required libraries:

```
from torch import autocast
from diffusers import DiffusionPipeline
```

4. Define the generator and set the seed to ensure reproducibility:

```
# Setting a seed would ensure reproducibility of the experiment.
generator = torch.Generator(device="cuda").manual_seed(42)
```

5. As we have seen in previous chapters, huggingface (and by extension `diffusers`) wraps models in the form of pipelines that are easy to use for the end user. Let's define the Hugging Face pipeline:

```
# Define the Stable Diffusion pipeline
pipeline = DiffusionPipeline.from_pretrained(
    "CompVis/stable-diffusion-v1-4",
    torch_dtype=torch.float16,
)

# Set the device for the pipeline
pipeline = pipeline.to("cuda")
```

6. Pass a text prompt through the pipeline defined above:

```
prompt = "baby in superman dress"
image = pipeline(prompt, generator=generator)
```

7. Visualize the image:

```
image.images[0]
```

This gives the following output:

Figure 16.36: Image generated by Stable Diffusion

Note that the above generated image has multiple issues – multiple babies are generated, legs do not seem aligned, etc. Can we do better?

8. Stable Diffusion has an XL variant that has been trained on more data while generating higher-resolution images (1024x1024), due to which the chances of such mistakes are lower. Let's replace the base model with the XL version of it, as follows:

```
# Define the Stable Diffusion XL pipeline
pipeline = DiffusionPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    torch_dtype=torch.float16,
)
# Set the device for the pipeline
pipeline = pipeline.to("cuda")
```

9. Pass the prompt through the pipeline that we just defined:

```
prompt = "baby in superman dress"
image = pipe(prompt, generator=generator)
```

The output is:

Figure 16.37: Output of SDXL

Note that, the image generated using the XL pipeline is much better than the image generated using the base pipeline. However, can we get even better?

10. Prompt engineering comes to the rescue in this scenario. We can modify our prompt as follows:

```
prompt = "baby in superman dress, photorealistic, cinematic"
```

The preceding prompt, when passed through the XL pipeline, results in an output as follows:

Figure 16.38: Realistic output of SDXL

Note that, by adding words like “photorealistic” and “cinematic”, we were able to generate an image that looks more dramatic than the plain one generated earlier.

Summary

In this chapter, we learned how CLIP helps in aligning embeddings of both text and images. We then gained an understanding of how to leverage the SAM to perform segmentation on any image. Next, we learned about speeding up the SAM using FastSAM. Finally, we learned about leveraging diffusion models to generate images both unconditionally and conditionally given a prompt.

We covered sending different modalities of prompts to the segment-anything model, tracking objects using the SAM, and combining multiple modalities using `ImageBind` in the associated GitHub repository.

With this knowledge, you can leverage the foundational models on your data/tasks with very limited/no training data points, such as training/leveraging models for the segmentation/object detection tasks that we learned about in *Chapters 7 to 9* with minimal/no data.

In the next chapter, you will learn about tweaking diffusion models further to generate images of interest to you.

Questions

1. How are text and images represented in the same domain using CLIP?
2. How are different types of tokens, such as point tokens, bounding box tokens, and text tokens, calculated in Segment Anything architecture?
3. How do diffusion models work?
4. What makes Stable Diffusion different from normal diffusion?
5. What is the difference between Stable Diffusion and the SDXL model?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/modcv>

